

When Does Seasonal Decomposition Help? A Feature-Driven Meta-Learning Study for Time Series Forecasting

Diogo Costa^{1*}

^{1*}Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias,
Porto, 4200-465, Portugal.

Corresponding author(s). E-mail(s): diogocosta1104@gmail.com;

Abstract

Keywords: Time series forecasting, Seasonal decomposition, STL, Meta-learning, Feature engineering, AutoML, Global forecasting models

1 Introduction

2 Related Work

3 Methodology

3.1 Problem Formulation

Let $\mathbf{y} = (y_1, y_2, \dots, y_T)$ be a univariate time series with seasonal period m and forecast horizon h . Given a set of forecasting methods $\mathcal{M}$ and decomposition strategies $\mathcal{D}$, the question is whether observable properties of $\mathbf{y}$ can predict which pair $(d, M) \in \mathcal{D} \times \mathcal{M}$ minimises forecast error for that series.

To study this, we extract a six-dimensional feature vector $\phi(\mathbf{y})$, partition series into tercile groups along each feature dimension, train all (d, M) combinations within each group, and compare performance across groups. If decomposition is beneficial, this signal should appear as a systematic performance difference between feature terciles.

3.2 Datasets

We use two established benchmarks from the M-Competition series: the M3 [1] and M4 [2] datasets, restricted to monthly and quarterly frequencies. Table 1 summarises the resulting collection.

Table 1 Dataset summary. Horizon and season length follow the competition definitions; series counts reflect the representative sample used for benchmarking (see Section 3.2.1).

Dataset	Frequency	Season (m)	Horizon (h)	Series
M3 Monthly	Monthly	12	18	750
M4 Monthly	Monthly	12	18	750
M3 Quarterly	Quarterly	4	8	750
M4 Quarterly	Quarterly	4	8	750
Total				3,000

3.2.1 Representative Sampling.

The M3 and M4 pools each contain several thousand series per frequency, so we draw 750 series per source dataset using stratified quantile sampling. Series are ranked by each feature ϕ_k and divided into $S = 30$ equal-count strata; the 750-series quota is distributed across strata proportionally and filled by draws without replacement. This ensures the sample reproduces the marginal feature distribution of the full pool. A fixed random seed makes the sampling deterministic.

3.3 Time Series Feature Extraction

Six features characterise each series. Where noted, they operate on STL residuals (Section 3.4) to focus on intrinsic series behaviour rather than removable seasonal structure. All features are computed from the training observations only, with no access to the test period.

Non-normality (ϕ_1).

Residuals r_t from the STL fit are standardised as $z_t = (r_t - \tilde{r}) / (1.4826 \cdot \text{MAD}(r) + \varepsilon)$, where $\tilde{r}$ is the median. The D’Agostino–Pearson omnibus test [3] is applied to $\{z_t\}$, and its p -value is converted to a log-scale score:

$$\phi_1 = \min(-\log_{10}(p_1 + 10^{-12}), 12). \quad (1)$$

High values indicate strong departure from Gaussian residuals.

Nonlinearity (ϕ_2).

We assess whether lagged history contains nonlinear predictive structure by comparing a linear and a nonlinear autoregressive model. Let $\ell = \min(m, 12)$ be the embedding

dimension. We construct lag-feature matrices from $\{(y_{t-\ell}, \dots, y_{t-1}, y_t)\}$ and split the resulting dataset at three chronological split points (55%, 70%, and 85% of the series length) to obtain multiple train/test folds. On each fold we fit a Ridge regression [4] ($\alpha = 1$) and a Random Forest [5] (40 trees, max depth 4). The feature is the mean relative improvement of the nonlinear model:

$$\phi_2 = \min\left(\max\left(0, \frac{\overline{\text{MAE}}_{\text{linear}} - \overline{\text{MAE}}_{\text{RF}}}{\overline{\text{MAE}}_{\text{linear}} + 10^{-8}}\right), 2\right), \quad (2)$$

where $\overline{\text{MAE}}$ denotes the mean across folds. A value of zero indicates the Random Forest offers no improvement over ridge regression; this is structurally common in linear datasets, as discussed in Section 4.4.

Spectral entropy (ϕ_3).

We compute the power spectral density of the demeaned series via a periodogram and derive its normalised Shannon entropy:

$$\phi_3 = -\frac{\sum_k p_k \ln p_k}{\ln K} \in [0, 1], \quad (3)$$

where $p_k = \hat{S}(f_k) / \sum_j \hat{S}(f_j)$ are the normalised spectral weights and K is the number of non-zero frequencies. Low values indicate a spectrum concentrated at a few dominant frequencies (strong, regular periodicity); high values indicate a diffuse, unpredictable spectrum [6].

Evolving seasonality (ϕ_4).

To detect non-stationary seasonal behaviour, we compute the seasonal strength (Equation 4) in rolling windows of length $\max(36, 3m)$ for monthly and $\max(16, 4m)$ for quarterly series, advancing by $\lfloor m/2 \rfloor$ observations per step. The seasonal strength in each window is

$$F_s^{(w)} = \max\left(0, 1 - \frac{\text{Var}(R^{(w)})}{\text{Var}(S^{(w)} + R^{(w)})}\right), \quad (4)$$

where $S^{(w)}$ and $R^{(w)}$ are the STL seasonal and residual components in window w . The feature is the standard deviation of the sequence $\{F_s^{(w)}\}$: a high value indicates that seasonal strength changes substantially over time.

Structural break strength (ϕ_5).

We detect level shifts in the nonseasonal component $T_t + R_t$ using the PELT algorithm [7] with an RBF cost function and a $\log(n)$ -proportional penalty. Let $\mathcal{B}$ be the set of detected breakpoints in the central 60% of the series (to exclude boundary

effects). For each candidate break $b \in \mathcal{B}$, the mean-shift contrast is

$$\Delta_b = \frac{|\bar{x}_{<b} - \bar{x}_{\geq b}|}{1.4826 \cdot \text{MAD}(T + R) + \varepsilon}, \quad (5)$$

and $\phi_5 = \min(\max_b \Delta_b, 20)$. This measures the magnitude of the largest structural shift relative to the scale of the nonseasonal component.

Conditional heteroscedasticity (ϕ_6).

We test for ARCH effects [8] in the normalised STL residuals $\{z_t\}$ using Engle’s LM test with lag order $\min(12, \lfloor n/5 \rfloor)$ and return

$$\phi_6 = \min(-\log_{10}(p_6 + 10^{-12}), 12), \quad (6)$$

where p_6 is the LM test p -value. High values indicate serially correlated residual variance (volatility clustering).

3.3.1 Feature Bucket Assignment.

For each feature ϕ_k and frequency, all selected series are jointly ranked by their feature value and assigned to tercile buckets: *Low* (bottom third), *Medium* (middle third), and *High* (top third). The bucket boundaries are the 33.3rd and 66.7th percentiles computed across both M3 and M4 series combined. This partitioning drives the feature-conditioned finetuning strategy described in Section 3.6.

3.4 STL Decomposition Framework

Seasonal-Trend decomposition using Loess (STL) [9] factorises a time series as

$$y_t = T_t + S_t + R_t, \quad (7)$$

where T_t is a locally smooth trend, S_t is a periodic seasonal component (constrained to be approximately zero-mean within each period), and R_t is the remainder. We use the robust variant of STL (down-weighting outlier observations during the Loess smoothing passes), implemented via `statsmodels` [10].

We evaluate three strategies for exploiting this decomposition:

Direct (*without_stl*). The forecasting model operates directly on the raw series $\mathbf{y}$. No decomposition is applied.

STL-Seasonal-Naive (*stl_seasonal_naive*). The model is trained on the nonseasonal component $N_t = T_t + R_t$, producing a forecast $\hat{N}_{t+1:t+h}$. The seasonal component is extrapolated by repeating the last observed full seasonal cycle: $\hat{S}_{t+j} = S_{t+j-m}$ for $j = 1, \dots, h$. The final forecast is $\hat{y}_{t+j} = \hat{N}_{t+j} + \hat{S}_{t+j}$.

STL-All-Components (*stl_model_all_components*). Three separate model instances are trained on T_t , S_t , and R_t respectively, yielding component forecasts $\hat{T}$,

$\hat{S}$, and $\hat{R}$. The final forecast is the sum:

$$\hat{y}_{t+j} = \hat{T}_{t+j} + \hat{S}_{t+j} + \hat{R}_{t+j}, \quad j = 1, \dots, h. \quad (8)$$

3.5 Forecasting Model Families

We benchmark four families across two training paradigms.

Statistical (per-series).

AutoARIMA [11], AutoSARIMA (forced $D = 1$), and AutoETS [12], all implemented via statsforecast [13].

Machine learning (global).

AutoRidge, AutoLinearRegression, AutoXGBoost [14], and AutoRandomForest, trained jointly on all cohort series via sliding-window lag features (MLForecast [15], Optuna HPO [16], 100 trials).

Neural (global).

AutoNLinear, AutoNHITS [17], and AutoLSTM (NeuralForecast [18]).

Transformers (global).

AutoTFT [19] and AutoPatchTST [20] (NeuralForecast).

3.6 Bucket-Specialised Training

For global model families (ML, Neural, Transformers), we investigate whether *regime-specialised* global models outperform a single global model trained on all 750 series. The hypothesis is that series sharing the same feature-tercile level exhibit sufficiently homogeneous dynamics to benefit from a dedicated model.

For each task configuration (feature $\times$ frequency $\times$ decomposition $\times$ model), three independent global models are trained—one per tercile bucket (*Low*, *Medium*, *High*)—each using *exclusively* the ≈ 250 series in its bucket, and evaluated *exclusively* on the test series from the same bucket. Performance is measured as the finetuning delta $\Delta_{\text{FT}} = \text{RelNaive}_{\text{global}}^{(10)} - \text{RelNaive}_{\text{bucket}}^{(10)}$ (positive = specialised model wins). The global model (all 750 series, no bucket restriction) serves as the comparison baseline throughout.

Statistical models are excluded as they fit independently per series and are unaffected by the training-pool composition. For global families, each (decomposition, model, feature) configuration is evaluated twice: once with all 750 series as the global baseline, and once per tercile bucket under the specialised condition.

3.7 Evaluation Protocol

3.7.1 Cross-Validation.

We use rolling-origin cross-validation [21] with $W = 3$ windows per series, advancing by h observations per window. For each series i and window w , we define a training set ending at cutoff $c_i^{(w)}$ and a test set of exactly h future observations.

3.7.2 Metric.

The primary metric is the *Relative Naive* (RelNaive), defined as

$$\text{RelNaive} = \frac{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}})}{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}}^{\text{naive}})}, \quad (9)$$

where $\hat{\mathbf{y}}^{\text{naive}}$ is the seasonal naive forecast ($\hat{y}_{t+j}^{\text{naive}} = y_{t+j-m}$, $j = 1, \dots, h$). A value below 1 indicates the model outperforms the seasonal naive baseline. We report both unclipped (RelNaive) and clipped (RelNaive⁽¹⁰⁾, capped at 10) variants; the latter reduces the influence of catastrophic failures on aggregate rankings. Aggregate statistics are computed as the mean over all series and windows within each reporting group (feature, decomposition method, model family, training scope).

The *STL delta* Δ_{STL} measures the improvement from applying STL decomposition relative to the Direct strategy:

$$\Delta_{\text{STL}} = \text{RelNaive}_{\text{Direct}}^{(10)} - \text{RelNaive}_{\text{STL}}^{(10)}, \quad (10)$$

so a positive value indicates that STL decomposition reduces error. The *specialisation delta* Δ_{FT} measures the improvement from training a bucket-specialised model over the global baseline:

$$\Delta_{\text{FT}} = \text{RelNaive}_{\text{global}}^{(10)} - \text{RelNaive}_{\text{bucket}}^{(10)}, \quad (11)$$

with a positive value indicating that regime-specialised training outperforms the global model on the same series.

Statistical significance of aggregate deltas is assessed with the Wilcoxon signed-rank test [?] applied at the series level, with p -values corrected for multiple comparisons using the Holm–Bonferroni method [?].

4 Results

4.1 Global Baseline Performance

All 3,000 series (1,500 monthly, 1,500 quarterly) are evaluated over three rolling cross-validation windows. Table 2 ranks the global model configurations by mean RelNaive⁽¹⁰⁾ (all series, no bucket restriction). **AutoETS with Direct (no decomposition)** is the best single configuration (mean 0.888, median 0.791), and the only

one to beat the naïve forecast at the mean level without preprocessing. The next nine are dominated by STL-SN paired with statistical or transformer models.

Table 2 Top-10 global model configurations by mean RelNaive⁽¹⁰⁾ (lower is better). Direct = no decomposition; STL-SN = STL-Seasonal-Naïve; STL-AC = STL-All-Components.

Rank	Family	Model	Decomp.	Mean	Median
1	Statistical	AutoETS	Direct	0.888	0.791
2	Statistical	AutoARIMA	STL-SN	0.930	0.887
3	Neural	AutoNLinear	STL-SN	0.931	0.889
4	Statistical	AutoSARIMA	STL-SN	0.931	0.889
5	ML	AutoLinearRegression	STL-SN	0.933	0.893
6	ML	AutoRidge	STL-SN	0.933	0.889
7	Transformer	AutoPatchTST	STL-SN	0.933	0.891
8	Statistical	AutoETS	STL-SN	0.935	0.918
9	Transformer	AutoPatchTST	STL-AC	0.936	0.826
10	Transformer	AutoTFT	STL-SN	0.936	0.919

A persistent divergence between mean and median runs throughout the study: under STL decomposition the ML and Neural families have mean RelNaive⁽¹⁰⁾ > 1 (1.12–1.19) while their medians remain near or below 1. A heavy right tail of catastrophic failures on a minority of series pulls the mean above the naïve baseline even when most series benefit. Both statistics are therefore reported throughout.

4.2 Effect of STL Decomposition

Table 3 reports Δ_{STL} for the global model. Positive values indicate STL reduces error relative to Direct.

Table 3 STL delta Δ_{STL} by decomposition and model family (global model; mean and median over all series and windows; win rate = fraction of series where STL helps).

Decomposition	Family	Mean Δ	Median Δ	Win Rate
STL-AC	Transformer	+0.039	+0.141	68.5%
STL-SN	Transformer	+0.063	+0.091	67.9%
STL-AC	Neural	−0.119	+0.063	57.6%
STL-SN	Neural	−0.109	+0.027	55.4%
STL-AC	ML	−0.191	+0.023	52.5%
STL-SN	ML	−0.194	−0.002	49.5%
STL-AC	Statistical	−0.028	−0.085	39.8%
STL-SN	Statistical	−0.009	−0.109	37.1%

Transformers

benefit robustly from STL decomposition: both STL-SN and STL-AC yield positive mean *and* median deltas, with win rates near 68%. The improvement is strongest under STL-AC (median +0.141), suggesting that training separate global submodels for trend, seasonal, and residual components aligns well with PatchTST and TFT inductive biases.

Neural and ML models

show the mean–median divergence at its most pronounced. For Neural, the median is positive (STL-AC: +0.063; STL-SN: +0.027) but the mean is deeply negative (−0.119, −0.109): STL helps most series but fails badly on those with non-stationary seasonal patterns where the decomposition itself is unreliable. ML is worse: STL-SN mean −0.194, median essentially zero (−0.002).

Statistical models

are consistently *harmed* by both decomposition strategies (win rates $\approx$ 38–40%, negative mean and median). AutoARIMA, AutoSARIMA, and AutoETS already model trend and seasonality internally; applying STL preprocessing introduces redundant transformation and additional parameter uncertainty, degrading rather than improving performance.

Frequency effects are marginal: quarterly series show slightly larger median gains (STL-AC median +0.024 vs. +0.033 monthly) with no directional difference.

4.3 Bucket-Specific Finetuning

4.3.1 Overall Results

Across 1,449,738 series-window observations, the specialised models achieve median $\Delta_{FT} = +0.050$, mean = −0.029, win rate = 56.1%. The mean–median gap mirrors the pattern seen for STL: bucket specialisation benefits the majority but fails badly on a tail of series, dragging the mean negative.

Medium-bucket series show the most consistent improvement (median +0.059, win rate 56.8%); Low and High are comparable (median +0.048 and +0.046). The Medium advantage reflects that these are the modal-regime series—the specialised model is not pulled toward the extremes present in either tail.

4.3.2 Non-Linear vs. Linear Models: A Critical Split

The clearest split is between non-linear and linear models:

Non-linear models benefit because focusing training on $\approx$ 250 regime-homogeneous series improves generalisation within that regime without sacrificing coverage of the full distribution. Both mean and median are positive, so the gain is robust.

Linear models are harmed: $\approx$ 250 training points are insufficient for stable coefficient estimation, and the bucket-induced distribution shift worsens the problem. The sub-50% win rate (48.1%) confirms a directional harm, not noise; Section 4.5 verifies significance.

Table 4 Bucket-specific finetuning delta by model type (aggregated over all families, buckets, decompositions, and series).

Model type	Mean Δ_{FT}	Median Δ_{FT}	Win rate
Non-linear	+0.082	+0.092	60.1%
Linear	−0.251	−0.014	48.1%

Table 5 gives the per-model breakdown.

Table 5 Bucket-specific finetuning delta per model (aggregated over all buckets, decompositions, features, and frequencies). Models are classified as linear (L) or non-linear (NL).

Family	Model	Type	Mean Δ	Median Δ	Win rate
ML	AutoXGBoost	NL	+0.236	+0.187	65.3%
Neural	AutoLSTM	NL	+0.231	+0.162	64.2%
ML	AutoRandomForest	NL	+0.146	+0.155	61.6%
Neural	AutoNHITS	NL	−0.031	+0.076	59.2%
Transformer	AutoTFT	NL	−0.076	+0.059	57.5%
Transformer	AutoPatchTST	NL	−0.013	+0.014	52.6%
ML	AutoRidge	L	−0.327	−0.004	49.5%
ML	AutoLinearRegression	L	−0.327	−0.002	49.7%
Neural	AutoNLinear	L	−0.099	−0.029	45.1%

AutoXGBoost and AutoLSTM show the largest gains (median +0.187, win 65.3%; median +0.162, win 64.2% respectively), benefiting from the focused within-bucket training distribution. AutoNHITS and the transformers have positive medians but negative means, indicating fat-tailed distributions. AutoRidge and AutoLinearRegression suffer the largest harms (mean $\Delta = -0.327$), a direct consequence of coefficient instability on small training pools. **Bucket specialisation should be restricted to non-linear models.**

4.3.3 Interaction with STL Decomposition

Finetuning interacts differently with each decomposition strategy:

Table 6 Bucket finetuning delta by decomposition strategy (all non-statistical model families and buckets combined).

Decomposition	Mean Δ_{FT}	Median Δ_{FT}	Win rate
Direct	−0.156	+0.108	60.4%
STL-SN	+0.044	+0.032	55.6%
STL-AC	+0.026	+0.021	52.2%

Direct gives the highest median (+0.108) but the worst mean (−0.156): without seasonal preprocessing, a model trained on ≈ 250 series overfits to the idiosyncratic structure of its bucket. **STL-SN is the most robust pairing** (mean +0.044, median +0.032): removing seasonality first produces a more stationary target, reducing catastrophic failures.

4.4 The Feature Nonlinearity Edge Case

The `feature_nonlinearity` (ϕ_2) feature exposes a structural limitation of tercile-based bucketing. Because ϕ_2 is zero-floored (Equation 2), the M3/M4 series accumulate at exactly zero: 81.8% monthly, 92.8% quarterly. Both tercile boundaries therefore coincide at $q_{1/3} = q_{2/3} = 0$, so all zero-valued series fall into Low and all non-zero series into High, leaving the Medium bucket empty (Table 7).

Table 7 Bucket sizes for `feature_nonlinearity` (ϕ_2).
 $q_{1/3} = q_{2/3} = 0$ forces binary Low/High stratification.

Frequency	Low	Medium	High	Total
Monthly	1,227 (81.8%)	0	273 (18.2%)	1,500
Quarterly	1,392 (92.8%)	0	108 (7.2%)	1,500

The 27 affected task configurations (1.9% of all bucket tasks) are excluded from comparative analyses. The binary split retains a meaningful interpretation: Low = series where lagged ridge regression is near-optimal; High = the minority with genuinely exploitable nonlinear structure. This is in fact the sharpest regime contrast among all six features.

4.5 Statistical Significance

Wilcoxon signed-rank tests with Holm–Bonferroni correction were applied to each (model, decomposition, bucket) combination ($n = 81$ tests total). Results:

- **Significantly helps:** 55/81 (67.9%), all $p_{\text{Holm}} < 0.001$.
- **Significantly harms:** 23/81 (28.4%), all $p_{\text{Holm}} < 0.001$.
- **Neutral (not significant):** 3/81 (3.7%).

The five most improved combinations are all non-linear models paired with the Medium bucket and STL decomposition (Table 8). The STL-SN + Medium + AutoXGBoost combination is the single strongest result in the study (median $\Delta_{\text{FT}} = +0.334$, win rate 72.3%).

The significantly harmed combinations are exclusively linear models and AutoN-Linear, with the worst harm for AutoNLinear under STL-AC (median $\Delta_{\text{FT}} = -0.088$, win rate 34.5%). Linear models under STL-AC suffer doubly: the global model must simultaneously fit trend, seasonal, and residual components of ≈ 250 series, giving each linear submodel too few degrees of freedom while compounding errors through recomposition.

Table 8 Top-5 significantly improved model $\times$ bucket combinations (Wilcoxon + Holm, all $p_{\text{Holm}} < 0.001$).

Family	Model	Decomp.	Bucket	Med. Δ	Win rate
ML	AutoXGBoost	STL-SN	Medium	+0.334	72.3%
ML	AutoXGBoost	STL-AC	Medium	+0.308	66.9%
Neural	AutoLSTM	STL-SN	Medium	+0.299	69.2%
ML	AutoRandomForest	STL-AC	Medium	+0.291	64.4%
ML	AutoRandomForest	STL-SN	Medium	+0.290	68.2%

4.6 Summary of Findings

1. **STL decomposition effects are family-specific** (Section 4.2). Transformers benefit robustly (Δ_{STL} positive in mean *and* median, win rate $\approx 68\%$). Neural and ML models show positive medians but severely negative means due to STL failure on non-stationary series. Statistical models are uniformly harmed—their internal trend-seasonal modelling makes STL preprocessing redundant.
2. **Bucket specialisation improves non-linear models, harms linear ones** (Section 4.3.2). Non-linear architectures gain median +0.092 (win rate 60.1%); linear models lose median -0.014 (win rate 48.1%). The split is statistically significant in 67.9% of tested configurations.
3. **The Medium bucket is most reliable; STL-SN is the safest pairing** (Sections 4.3.1, 4.3.3). Medium-bucket series yield the most consistent specialisation gains. STL-SN before bucket training gives positive mean *and* median delta, eliminating the catastrophic failures seen under Direct.
4. **Feature nonlinearity collapses to binary bucketing on M-competition data** (Section 4.4). The zero-floor of ϕ_2 causes $q_{1/3} = q_{2/3} = 0$, emptying the Medium bucket and reducing stratification to Low/High for 1.9% of tasks. Tercile-based partitioning requires a minimum-support guard when features have degenerate distributions.

5 Discussion

6 Conclusion

Acknowledgements.

Declarations

- **Funding:** Not applicable.
- **Conflict of interest:** The author declares no competing interests.
- **Ethics approval:** Not applicable.
- **Data availability:** The M3 and M4 datasets are publicly available at <https://forecasters.org/resources/time-series-data/>.
- **Code availability:** Source code will be made available upon acceptance.

- **Author contribution:** D.C. conceived and designed the study, implemented the pipeline, ran all experiments, and wrote the manuscript.

References

- [1] Makridakis, S., Hibon, M.: The M3-Competition: Results, conclusions and implications. *International Journal of Forecasting* **16**(4), 451–476 (2000)
- [2] Makridakis, S., Spiliotis, E., Assimakopoulos, V.: The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting* **36**(1), 54–74 (2020)
- [3] D’Agostino, R.B.: An omnibus test of normality for moderate and large size samples. *Biometrika* **58**(2), 341–348 (1971)
- [4] Hoerl, A.E., Kennard, R.W.: Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* **12**(1), 55–67 (1970)
- [5] Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001)
- [6] Pincus, S.M.: Measuring complexity and predictability of time series with flexible multiscale entropy for sensor networks. *Sensors* **16**(6), 783 (2016)
- [7] Killick, R., Fearnhead, P., Eckley, I.A.: Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association* **107**(500), 1590–1598 (2012)
- [8] Engle, R.F.: Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica* **50**(4), 987–1007 (1982)
- [9] Cleveland, R.B., Cleveland, W.S., McRae, J.E., Terpenning, I.: STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics* **6**(1), 3–73 (1990)
- [10] Seabold, S., Perktold, J.: Statsmodels: Econometric and statistical modeling with Python. In: 9th Python in Science Conference, pp. 92–96 (2010)
- [11] Hyndman, R.J., Khandakar, Y.: Automatic time series forecasting: The **forecast** package for R. *Journal of Statistical Software* **27**(3), 1–22 (2008)
- [12] Hyndman, R.J., Koehler, A.B., Ord, J.K., Snyder, R.D.: *Forecasting with Exponential Smoothing: The State Space Approach*. Springer, ??? (2008)
- [13] Olivares, K.G., Challu, C., Garza, F., Mergenthaler, M., Nixtla: statsforecast: Lightning fast forecasting with statistical and econometric models. *arXiv preprint arXiv:2207.03175* (2022)
- [14] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *Proceedings*

- of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016)
- [15] Garza, F., Canseco, M.M., Challú, C., Olivares, K.G.: Mlforecast: Scalable machine learning for time series forecasting. arXiv preprint arXiv:2207.03175 (2022)
 - [16] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2623–2631 (2019)
 - [17] Challu, C., Olivares, K.G., Oreshkin, B.N., Garza, F., Mergenthaler-Canseco, M., Dubrawski, A.: N-HiTS: Neural hierarchical interpolation for time series forecasting. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 6989–6997 (2023)
 - [18] Olivares, K.G., Challu, C., Garza, F., Canseco, M.M., Dubrawski, A.: Neural-forecast: User friendly state-of-the-art neural forecasting models. arXiv preprint arXiv:2207.03175 (2023)
 - [19] Lim, B., Arik, S.Ö., Loeff, N., Pfister, T.: Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* **37**(4), 1748–1764 (2021)
 - [20] Nie, Y., H. Nguyen, N., Sinthong, P., Kalagnanam, J.: A time series is worth 64 words: Long-term forecasting with transformers. In: International Conference on Learning Representations (2023)
 - [21] Tashman, L.J.: Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting* **16**(4), 437–450 (2000)